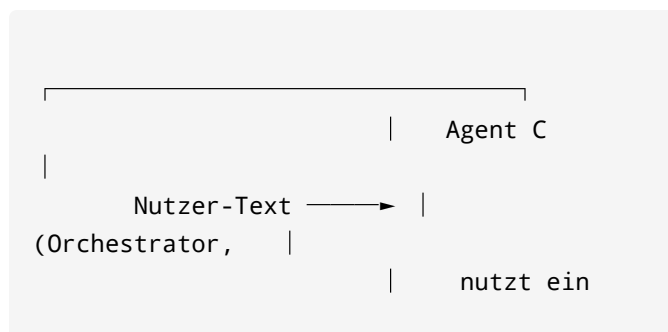


Tutorial (Teil 2): Ein LLM-Agent wählt zwischen zwei A2A-Agenten aus

Stand: September 2026, Fortsetzung von "Zwei Agenten per A2A kommunizieren lassen".
Basiert weiterhin auf `a2a-sdk 1.0.x`.

0. Was wir bauen

Wir erweitern das erste Tutorial um einen dritten Agenten: einen **Orchestrator** ("Agent C"), der selbst kein festes Verhalten hat, sondern ein kleines LLM benutzt, um zu entscheiden, welcher der beiden Spezial-Agenten eine eingehende Anfrage übernehmen soll.





Ablauf:

1. Agent C holt sich beim Start die **AgentCard** von Agent B **und** Agent D.
2. Kommt eine Nutzeranfrage rein, schickt Agent C die Anfrage zusammen mit den Skill-Beschreibungen beider Agenten an ein LLM.
3. Das LLM antwortet mit einer Entscheidung: "greet" oder "add" (plus ggf. extrahierte Parameter, z. B. die beiden Zahlen).

4. Agent C ruft darauf den passenden Agenten per A2A auf und gibt dessen Antwort zurück.

Das ist das **Orchestrator-Muster**, das im ersten Tutorial unter Punkt 9 bereits angedeutet wurde – jetzt aber mit echter, LLM-basierter Entscheidungslogik statt fest verdrahteter Auswahl.

1. Neuer Grundbegriff

Begriff	Bedeutung
Routing	Die Entscheidung, welcher nachgelagerte Agent eine Anfrage bearbeiten soll. Kann regelbasiert (if/else) oder LLM-basiert sein.
Agent-as-Tool	Muster, bei dem der Orchestrator die A2A-Agenten intern wie Werkzeuge behandelt, die das LLM "aufrufen" kann – die Entscheidung selbst bleibt aber beim LLM, nicht bei starrem Code.

2. Voraussetzungen

Zusätzlich zu Teil 1 brauchst du:

- Einen Anthropic API-Key (Umgebungsvariable `ANTHROPIC_API_KEY`)
- Das Python-Paket `anthropic`

```
cd a2a-tutorial
source .venv/bin/activate
pip install anthropic
export ANTHROPIC_API_KEY="dein-key-hier"
```

3. Projektstruktur danach

```
a2a-tutorial/
├── agent_b_server.py      # aus Teil
1: Greeter, Port 9999
├── agent_d_server.py      # NEU:
Adder, Port 9998
├── agent_c_orchestrator.py # NEU: LLM-
Router
└── agent_a_client.py      # aus Teil
1, optional weiter nutzbar
```

4. Agent D bauen: der "Adder"

Fast identisch zu Agent B aus Teil 1, nur mit anderer Logik und anderem Port.

Lege `agent_d_server.py` an:

```
import re
import uvicorn
```

```

from starlette.applications import
Starlette
from a2a.server.agent_execution import
AgentExecutor, RequestContext
from a2a.server.events import EventQueue
from a2a.server.request_handlers import
DefaultRequestHandler
from a2a.server.routes import
create_agent_card_routes,
create_jsonrpc_routes
from a2a.server.tasks import
InMemoryTaskStore, TaskUpdater
from a2a.types import (
    AgentCapabilities, AgentCard,
    AgentInterface, AgentSkill, TaskState,
)
from a2a.helpers import
new_task_from_user_message,
new_text_message, \
    new_text_part, get_message_text

class AdderAgent:
    """Die 'Intelligenz' von Agent D:
    addiert zwei Zahlen aus dem Text."""

    async def invoke(self, user_request:
str) -> str:
        zahlen = re.findall(r"-?\d+
(?:\.\d+)?", user_request)
        if len(zahlen) < 2:
            return "Ich brauche zwei
Zahlen, um sie zu addieren."
        a, b = float(zahlen[0]),

```

```

float(zahlen[1])
    ergebnis = a + b
    return f"{a} + {b} = {ergebnis}"

class AdderAgentExecutor(AgentExecutor):
    def __init__(self) -> None:
        self.agent = AdderAgent()

    async def execute(self, context:
RequestContext, event_queue: EventQueue) ->
None:
        if context.current_task:
            task = context.current_task
        else:
            task =
new_task_from_user_message(context.message)
            await
event_queue.enqueue_event(task)

            updater =
TaskUpdater(event_queue=event_queue,
task_id=task.id,

context_id=task.context_id)
            await updater.update_status(

state=TaskState.TASK_STATE_WORKING,

message=new_text_message("Rechne..."),
            )

            query =
get_message_text(context.message)

```

```

        result = await
self.agent.invoke(user_request=query or "")

        await updater.add_artifact(
            parts=
[new_text_part(text=result,
media_type="text/plain")]
        )
        await updater.update_status(

state=TaskState.TASK_STATE_COMPLETED,

message=new_text_message("Fertig!"),
        )

        async def cancel(self, context:
RequestContext, event_queue: EventQueue) ->
None:
            raise
NotImplementedError("Abbrechen wird hier
nicht unterstützt")

if __name__ == "__main__":
    skill = AgentSkill(
        id="add",
        name="Addieren",
        description="Addiert zwei Zahlen,
die im Text der Nachricht vorkommen.",
        input_modes=["text/plain"],
        output_modes=["text/plain"],
        tags=["demo", "math", "addition"],
        examples=["Addiere 3 und 5", "Was
ist 12 + 30?"],

```

```

    )

    agent_card = AgentCard(
        name="Agent D - Adder",
        description="Ein einfacher Rechen-
Agent, der zwei Zahlen addiert.",
        version="0.1.0",
        default_input_modes=["text/plain"],
        default_output_modes=
["text/plain"],

capabilities=AgentCapabilities(streaming=Tr
ue),
        supported_interfaces=[
            AgentInterface(
                protocol_binding="JSONRPC",

url="http://127.0.0.1:9998",
                protocol_version="1.0",
            )
        ],
        skills=[skill],
    )

    request_handler =
DefaultRequestHandler(

agent_executor=AdderAgentExecutor(),
        task_store=InMemoryTaskStore(),
        agent_card=agent_card,
    )

    routes = []

```

```
routes.extend(create_agent_card_routes(agent_card))

routes.extend(create_jsonrpc_routes(request_handler, "/"))
app = Starlette(routes=routes)
uvicorn.run(app, host="127.0.0.1", port=9998)
```

Starte ihn in einem eigenen Terminal:

```
python agent_d_server.py
```

Agent B aus Teil 1 sollte parallel in einem zweiten Terminal weiterlaufen (Port 9999).

5. Agent C bauen: der LLM-Orchestrator

Das ist der eigentlich neue Teil. Agent C ist selbst **kein** A2A-Server in diesem Beispiel, sondern nur ein Client, der zwei andere A2A-Server anspricht – du könntest ihn aber genauso gut zusätzlich als eigenen A2A-Server verpacken (mit eigener AgentCard), damit *er selbst* wieder von außen ansprechbar ist.

Für den Anfang bleiben wir beim einfacheren Fall: ein Skript, das man startet.

Lege `agent_c_orchestrator.py` an:

```
import asyncio
import json
import httpx
import anthropic

from a2a.client import A2ACardResolver,
ClientConfig, create_client
from a2a.helpers import new_text_message
from a2a.types import Role,
SendMessageRequest

AGENT_URLS = [
    "http://127.0.0.1:9999", # Agent B -
Greeter
    "http://127.0.0.1:9998", # Agent D -
Adder
]

llm_client = anthropic.Anthropic()

async def lade_agent_cards(httpx_client:
httpx.AsyncClient) -> list[dict]:
    """Holt die AgentCard jedes bekannten
Agenten und packt die
    relevanten Infos in eine einfache Liste
von Dicts."""
    cards = []
```

```

        for url in AGENT_URLS:
            resolver =
A2ACardResolver(httpx_client=httpx_client,
base_url=url)
            card = await
resolver.get_agent_card()
            cards.append({
                "url": url,
                "name": card.name,
                "skills": [
                    {"id": s.id, "description":
s.description, "examples": s.examples}
                    for s in card.skills
                ],
            })
        return cards

```

```

def baue_routing_prompt(nutzer_text: str,
cards: list[dict]) -> str:
    skill_uebersicht = json.dumps(cards,
ensure_ascii=False, indent=2)
    return f""Du bist ein Router für
Agenten-Anfragen.

```

Verfügbare Agenten und ihre Fähigkeiten:
{skill_uebersicht}

Nutzeranfrage: "{nutzer_text}"

Antworte AUSSCHLIESSLICH mit einem JSON-Objekt in dieser Form,
ohne zusätzlichen Text und ohne Markdown-Codeblock:

```

{{"url": "<URL des zuständigen Agenten>",
"skill_id": "<id des Skills>"}}
"""

def
entscheide_zustaendigen_agenten(nutzer_text
: str, cards: list[dict]) -> dict:
    prompt =
baue_routing_prompt(nutzer_text, cards)
    response = llm_client.messages.create(
        model="claude-sonnet-4-6",
        max_tokens=200,
        messages=[{"role": "user",
"content": prompt}],
    )
    text = response.content[0].text.strip()
    return json.loads(text)

async def main():
    nutzer_text = input("Was möchtest du
senden? ")

    async with httpx.AsyncClient() as
httpx_client:
        # 1. AgentCards aller bekannten
Agenten laden
        cards = await
lade_agent_cards(httpx_client)

        # 2. LLM entscheiden lassen, wer
zuständig ist
        entscheidung =

```

```

entscheide_zustaendigen_agenten(nutzer_text
, cards)
    ziel_url = entscheidung["url"]
    print(f"\n[Orchestrator] LLM hat
entschieden: {entscheidung}")

    # 3. Passenden Agenten per A2A
ansprechen
    resolver =
A2ACardResolver(httpx_client=httpx_client,
base_url=ziel_url)
    ziel_card = await
resolver.get_agent_card()

    config =
ClientConfig(streaming=False)
    client = await
create_client(agent=ziel_card,
client_config=config)

    message =
new_text_message(nutzer_text,
role=Role.ROLE_USER)
    request =
SendMessageRequest(message=message)

    print(f"[Orchestrator] Sende an
{ziel_card.name} weiter...\n")
    async for chunk in
client.send_message(request):
        print(chunk)

```

```
if __name__ == "__main__":  
    asyncio.run(main())
```

Was hier neu ist gegenüber Teil 1

- **lade_agent_cards** : Agent C kennt die *URLs* von Agent B und D fest (Punkt 1 aus der letzten Antwort – "direkt bekannte URL"), holt sich daraus aber dynamisch die aktuellen Skill-Beschreibungen. Fällt einer der Agenten weg oder ändert er seine Skills, merkt Agent C das beim nächsten Start.
- **baue_routing_prompt** : die Skill-Beschreibungen aus den AgentCards werden direkt ins Prompt gepackt – das LLM sieht praktisch dieselbe "Werbetafel", die vorher rein informativ war, und nutzt sie jetzt aktiv zur Entscheidung.
- **entscheide_zustaendigen_agenten** : ein ganz normaler LLM-Aufruf, der strukturiertes JSON zurückgeben soll. Das ist bewusst simpel gehalten – in Produktion würdest du hier **tool_use** mit einem festen JSON-Schema nutzen, um die Antwort zuverlässiger zu parsen.
- Erst **nach** der Entscheidung des LLMs wird der eigentliche A2A-Aufruf

gemacht – der Orchestrator selbst führt keine Fachlogik aus, er delegiert.

6. Ausprobieren

Drei Terminals parallel laufen lassen:

```
# Terminal 1
python agent_b_server.py

# Terminal 2
python agent_d_server.py

# Terminal 3
python agent_c_orchestrator.py
```

Im dritten Terminal z. B. eingeben:

```
Was möchtest du senden? Kannst du 7 und 15
addieren?
```

Erwartete Ausgabe (sinngemäß):

```
[Orchestrator] LLM hat entschieden: {"url":
"http://127.0.0.1:9998", "skill_id": "add"}
[Orchestrator] Sende an Agent D - Adder
weiter...

task {
  ...
  artifacts { parts { text: "7.0 + 15.0 =
```

```
22.0" } }  
}
```

Probiere danach im selben Terminal (neu starten)
auch:

```
Was möchtest du senden? Sag mir Hallo!
```

Diesmal sollte das LLM Agent B auswählen.

7. Was ist da technisch passiert?

1. Agent C hat sich beim Start die AgentCards von Agent B und Agent D geholt – klassische A2A-Discovery wie in Teil 1, nur für zwei Agenten statt einem.
2. Statt selbst per `if/else` zu entscheiden ("enthält der Text Zahlen? dann Adder"), wurde die Entscheidung an ein LLM delegiert, dem die Skill-Beschreibungen aus den AgentCards als Kontext mitgegeben wurden.
3. Das LLM hat eine strukturierte Entscheidung (JSON) zurückgegeben.
4. Erst danach ist der eigentliche A2A-Request an den ausgewählten Agenten gegangen – exakt derselbe Mechanismus (Card

abrufen, Client erzeugen,
`send_message`) wie in Teil 1.

Der Unterschied zu einer reinen MCP-Tool-Auswahl:
Die "Tools" hier sind
komplette, unabhängig laufende Agenten mit eigener
Task-Lifecycle, die auch
auf einer anderen Maschine, in einer anderen Sprache
oder mit eigener
interner Logik laufen könnten. Das LLM im
Orchestrator wählt nur *aus*, es
ruft keine Funktion direkt auf – die eigentliche
Ausführung bleibt beim
Ziel-Agenten.

8. Nächste Schritte

- **Agent C selbst als A2A-Server verpacken**, mit
eigener AgentCard
("Orchestrator, kann grüßen und addieren"), damit
man ihn wiederum aus
Teil 1 heraus mit `agent_a_client.py` ansprechen
kann – ganz ohne dass
Agent A weiß, dass dahinter zwei Sub-Agenten
stecken.
- **Mehr als zwei Agenten**: die Liste `AGENT_URLS`
einfach erweitern; das
Routing-Prompt skaliert ohne Codeänderung mit.

- **tool_use** statt Freitext-JSON: die Routing-Entscheidung über ein festes Tool-Schema erzwingen, damit das Parsen robuster wird.
- **Parameter-Extraktion durch das LLM:** das LLM nicht nur den Ziel-Agenten wählen lassen, sondern auch gleich die extrahierten Werte (z. B. die beiden Zahlen) strukturiert mitliefern, statt sie im Ziel-Agenten per Regex aus dem Text zu holen.